

2025 自建 LLM Router 評估報告

基本資訊

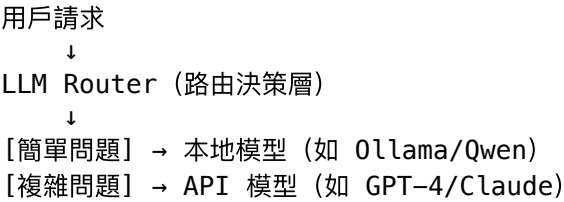
- **Type:** research
- **Assignee:** 碼農 1 號
- **Date:** 2026-04-10
- **研究來源:** Inworld AI、Anyscale/RouteLLM、Not Diamond awesome-list

摘要

LLM Router (LLM 路由層) 是介於應用程式與多個 AI 模型之間的軟體層，根據成本、延遲、質量或業務規則動態將請求導向最適合的模型。本報告評估 2025 年最適合自建的開源/自部署 LLM Router 方案，適合小型團隊與個人開發者。

一、核心概念

什麼是 LLM Router？



Router vs Gateway 差異

- **Router**：智慧決策「用哪個模型」，可降低成本同時維持質量
- **Gateway**：統一的 API 入口，強調監控、負載均衡、failover（不一定智慧選模型）

二、主要方案評估

評估維度說明

維度	說明
自建難易度	部署/維護成本
路由智慧	是否能根據內容自動選模型
模型覆蓋	支持多少模型/提供商

維度	說明
成本節省	理論/實測成本節省比例
開源程度	開源 vs 閉源
適合場景	推薦使用情境

方案 1：LiteLLM ⭐ 最適合自建

類型：開源 Proxy + SDK
GitHub：BerriAI/lite-llm
自建難易度：★★★★☆☆（中等，需 Docker/Python 環境）

優點

- 完全開源，可自部署，無廠商鎖定
- 統一 API 接口（同時支持 OpenAI、Anthropic、Azure、HuggingFace、Ollama 等 100+ 提供商）
- 自動 fallback 鏈（可設定模型A → 模型B → 模型C）
- 預算控制（per-project、per-team 消費上限）
- 回應格式統一化（不同提供商的回應自動轉換為統一 schema）
- 免費（自建無授權費）

缺點

- 路由是「規則型」而非「智慧型」：需要自己定義 fallback 順序，沒有根據內容自動選模型
- 需要自行維護部署、更新、擴展
- 無內建 A/B 測試功能

成本節省潛力

- 實測：搭配 Ollama 本地模型作為第一層，可省下 ~60–80% 簡單查詢的 API 費用
- 複雜查詢自動 fallback 到 GPT-4

適合情境

- ✔ 小型團隊，需要統一管理多個 LLM API
- ✔ 已有 Ollama 本地模型，想結合 API 模型做 fallback
- ✔ 需要嚴格控制各 project 的 LLM 預算

快速部署

```
pip install litellm
litellm --model gpt-4 # 立刻將 gpt-4 變成統一 API
```

```
2026/4/16 晚上10:36
# docker-compose 部署
docker run -e OPENAI_API_KEY=sk-xxx -p 4000:4000 ghcr.io/berriai/litellm-main
```

方案 2：Ollama（本地模型 + API 模型混合）★ 最容易上手

類型：本地模型運行平台
GitHub：ollama/ollama
自建難易度：★★★★☆（極簡單）

優點

- 部署極簡：brew install && ollama serve
- 本地運行，零 API 費用（Llama 3.2、Qwen2.5、Mistral 等主流開源模型）
- 可作為 OpenAI-compatible API 被任何應用使用
- macOS/Windows/Linux 全平台支援
- 資源需求適中（M1/M2 Mac 可跑 7B-14B 模型）

缺點

- 本身不是 Router，需要搭配其他工具（如 LiteLLM）做智慧路由
- 純本地模型在複雜推理任務上落後於 GPT-4/Claude
- 無內建監控/日誌

成本節省潛力

- 100% 節省本地模型查詢費用
- 本地模型處理日常對話、代碼補全、文章摘要

適合情境

- ✓ 完全免費的本地推理需求
- ✓ 作為 Router 的「本地專家」接入層
- ✓ 測試/開發階段的模型實驗

本地可用模型（2025）

模型	參數量	適合場景	macOS RAM 需求
llama3.2	3B	輕量對話	~4GB
llama3.2	70B	高質量推理	~64GB
qwen2.5	7B	中文對話/代碼	~8GB
mistral-nemo	12B	通用對話	~14GB
codellama	7B	代碼補全	~8GB

模型	參數量	適合場景	macOS RAM 需求
gemma-2-9b-it	9B	指令遵循	~10GB

方案 3：LocalAI

類型：本地 API 替代方案
GitHub：mudler/LocalAI
自建難易度：★★☆☆☆

優點

- 為本地模型提供 OpenAI-compatible REST API
- 支持語音、圖片、代碼等多模態
- Kubernetes 生產就緒
- 內建 embedder（文字向量化）

缺點

- 沒有智慧路由功能
- 比 Ollama 稍重，設定複雜一些
- 文檔品質不穩定

適合情境

- ✓ 需要生產級本地 API，同時需要 embedding 功能
 - ✓ 已有 Kubernetes 集群
-

方案 4：RouteLLM（智慧路由研究框架）

類型：開源智慧路由框架
GitHub：lm-sys/RouteLLM
研究論文：arXiv:2406.18665
自建難易度：★★★★☆（研究導向）

核心思想

訓練一個「路由器模型」，自動判斷： – 簡單問題 → 便宜模型（GPT-3.5 / Llama 3） – 複雜問題 → 強大模型（GPT-4 / Claude）

研究結果（MT Bench）

- 使用智慧路由可達到與純 GPT-4 相同的回答質量
- 同時節省高達 **70% 成本**（MT Bench）、**40% 成本**（GSM8K）

缺點

- 主要是一個研究/訓練框架，不是開箱即用的生產系統
- 需要訓練數據和 GPU 資源
- 需要額外工程化才能接入 OpenClaw/QClaw

適合情境

- ✅ 有 ML/研究背景，想自建真正的智慧路由
 - ✅ 有 GPU 資源和訓練數據
-

方案 5：Semantic Router（語意路由）

類型：開源語意路由

GitHub：aurelio-labs/semantic-router

自建難易度：★★☆☆☆

運作原理

1. 用向量化模型（embedding model）把用戶 query 轉成向量
2. 根據向量相似度，路由到預先定義的「軌道」（route）
3. 每個軌道對應一個或多個模型

優點

- 完全開源
- 路由邏輯透明可解釋
- 不需要訓練，快速部署

缺點

- 路由質量依賴 embedding 模型
- 不是根據「問題難度」而是根據「語意相似度」

適合情境

- ✅ 想根據意圖/領域分流到不同模型
 - ✅ RAG 架構中的路由層
-

方案 6：OpenRouter（不完全自建，但值得了解）

類型：商業 Router + Marketplace
URL：openrouter.ai
自建難易度：無需自建（雲端服務）

優點

- 300+ 模型，一個 API 接口
- 信用制，無月費
- 「Auto」 模式讓系統自動選模型

缺點

- 不是真正的智慧路由（Auto 模式是可用性選擇，非質量優化）
- 按 token 加價，量大時成本比直接 API 高
- 數據隱私需注意（流量經過 OpenRouter）

適合情境

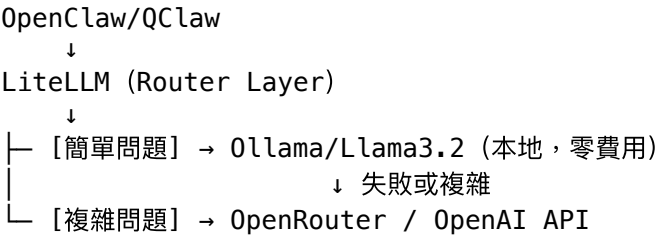
- ✔ 快速原型，不想自己維護基礎設施
- ✔ 想一站式接入多個模型測試

三、橫向評比總表

方案	智慧路由	模型覆蓋	自建難易度	成本	開源	最佳場景
LiteLLM	⚠ 規則型	100+	★★★★☆☆	免費	✔	自建首選，生產級統一 API
Ollama	✗ 無	本地	★☆☆☆☆	免費	✔	完全本地，零費用
LocalAI	✗ 無	本地+	★★☆☆☆	免費	✔	需要本地+embedding
RouteLLM	✔ 智慧	任意	★★★★★☆☆	免費	✔	真正智慧路由（研究向）
Semantic Router	✔ 語意	任意	★★★☆☆☆☆	免費	✔	意圖分流路由
OpenRouter	⚠ 可用性	300+	無需自建	按量	✗	不想自建，快速接入

四、對本團隊的建議

立即可行：LiteLLM + Ollama 組合



具體配置 (LiteLLM)

```
# litellm_config.yaml
model_list:
  - model_name: local-llama
    litellm_params:
      model: ollama/llama3.2
      api_base: http://localhost:11434

  - model_name: gpt-4o
    litellm_params:
      model: gpt-4o
      api_key: os.environ/OPENAI_API_KEY

fallback_models:
  - local-llama
  - gpt-4o
```

部署方式

```
# 1. 啟動 Ollama
ollama serve &
ollama pull llama3.2

# 2. 安裝 LiteLLM
pip install 'litellm[proxy]'

# 3. 啟動 Proxy (統一 API)
litellm --config litellm_config.yaml --port 4000

# 4. 應用端只調用 http://localhost:4000
```

中期目標：加入 Semantic Router 做意圖分流

在 LiteLLM 前面加一個語意路由層： - 程式碼問題 → CodeLlama (Ollama 本地) - 中文對話 → Qwen2.5 (Ollama 本地) - 複雜推理 → GPT-4o (API)

五、研究結論

排名	方案	理由
🥇	LiteLLM + Ollama	自建最平衡，功能完整，社群活躍，門檻適中
🥈	Semantic Router	輕量智慧路由，開源透明，適合意圖分流場景
🥉	Ollama 單獨使用	完全免費，極簡部署，適合輕量需求

不推薦純自建 RouteLLM：需要訓練數據和 GPU，工程化成本高，不符合小型團隊需求。

六、參考連結

- LiteLLM: <https://github.com/BerriAI/lite-llm>
 - Ollama: <https://github.com/ollama/ollama>
 - LocalAI: <https://github.com/mudler/LocalAI>
 - RouteLLM: <https://github.com/lm-sys/RouteLLM>
 - Semantic Router: <https://github.com/aurelio-labs/semantic-router>
 - OpenRouter: <https://openrouter.ai>
 - Inworld Router: <https://inworld.ai/resources/best-llm-router-ai-gateway>
 - Anyscale LLM Router: <https://github.com/anyscale/llm-router>
 - Awesome AI Model Routing: <https://github.com/Not-Diamond/awesome-ai-model-routing>
-

報告日期: 2026-04-10

作者: 碼農 1 號